

Introduction

- Être un utilisateur déclaré
- Connexion: id + mot de pass
- dialoguer avec le système via un interpréteur de commandes: **shell**.
- Le **shell** est: est un **interpréteur de commande**
c'est Aussi
un **Langage de programmation**

Le shell interpréteur de commande

- **Algorithme**

Principe :

Boucle_infinie

affichage du prompt (\$) d'attente clavier

lecture d'une commande (RETURN)

analyse syntaxique (découpe en mots)

substitution (caractères spéciaux)

exécution de la commande

Fin_de_Boucle

- **Rappel Redirections**

À la connexion, le Shell dispose de 3 flux de communication

l'entrée standard : `stdin (0)` -- > Clavier

la sortie standard : `stdout (1)` -- > Ecran

la sortie des erreurs : `stderr (2)` -- > Ecran

- **Redirection de la sortie Standard**

Permet de récupérer le résultat d'une commande dans un fichier

commande > nom_de_fichier

Fichier de redirection créé avant exécution de la commande (donc même si celle-ci échoue)

Si le fichier de redirection existe, son contenu est écrasé

- Pour ne pas écraser le fichier, on utilise la double redirection:

commande >> nom_de_fichier

Rq: Si le fichier n'existe pas, il est créé, comme pour une redirection simple

- **Redirection de la sortie Standard Erreur**

Elle est symbolisée par:

commande 2> fichier

Pour ne pas écraser le fichier, on utilise la double redirection:

commande 2> > nom_de_fichier

- **Combinaison des redirections**

Redirection de l'erreur vers la sortie standard :

commande > fichier 2>&1

L'ordre des redirections est important

commande > fichier 2>&1

n'est pas équivalent à

commande 2>&1 > fichier

- **Traitement des redirections**

Le Shell lit les commandes, les analyse, effectue les substitutions et exécute les actions

Mais il traite les redirections en tout premier

Exécution de **commande > fich** : le fichier fich est créé (ou vidé s'il existait) même si la commande n'existe pas ou échoue

commande > fich est équivalent à **> fich commande**

Redirection possible avec &n (n : descripteur)

Exemple :

cmde > f1 2>&1 (stdout et stderr dans f1)

Possibilité de fermeture avec
<&- (stdin)

>&- (stdout)

2>&- (stderr)

- **Tubes**

Tube (pipe) = zone mémoire permettant à deux processus de communiquer tout en étant synchronisés :

écrivain attend une place disponible

lecteur attend une information prête

En shell, cela permet de faire agir une commande sur le résultat d'une autre sans passer par des fichiers intermédiaires

Symbole | placé entre deux commandes redirige la sortie de la 1ère sur l'entrée de la deuxième

```
$ ls -l | grep -c '^d' # Nombre de répertoire
```

Le tube évite des E/S disques plus coûteuses :

\$ ls -l > rep # Création d'un fichier temporaire

\$ wc -l rep # Traitement de ce fichier
6

\$ rm rep # Suppression du fichier

Script shell

- Ensemble de commandes dans un fichier texte
- Les commandes sont regroupées par une syntaxe de langage de commande.
- Rendre le fichier exécutable
chmod a+x fichier

Les variables

Variables

Variables non typées

Affectation : nom=contenu (sans espace)

Utilisation : \$nom

\$ mois=Janvier

\$ echo \$mois

Janvier

(sont des variables locales 'par rapport au processus courant')

Affichage : set ou env

Suppression : unset var1

Portée des variables

Les variables sont locales au processus dans lequel elles ont été créées

Pour les rendre "globales" c-à-d les mettre dans l'environnement, il faut les exporter :

export var1

(la rendre accessible par tous les processus)

Ainsi, quand le shell crée un processus, celui-ci hérite de cet environnement.

Exemple : soit un fichier f1 exécutable contenant la commande ci-dessous :

```
echo "Dans f1 : a = $a"
```

Exécution :

a=oui affectation de a dans le père

./f1 exécution de f1

--> Affichage: Dans f1 : a =

export a a est exportée

./f1 ré-exécution de f1

--> Affichage: Dans f1 : a = oui

Variables prédéfinies

HOME : Répertoire de connexion

cd = cd \$HOME

MAIL : Nom de la boîte aux lettres

MAILCHECK: Nbre en seconde spécifiant la fréquence selon laquelle le fichier \$MAIL est surveillé

Commande interne/externe:

Internes : sont des commandes intégrées dans le binaire du programme *shell*
(exemple : cd, echo) (leur effet est limité au processus)

Externes : sont des binaires indépendants du *shell* (exemple : ls, cp)

PATH : répertoires de recherche des commandes **externes**

Ajouter rep au PATH : PATH=\$PATH:rep

L'ordre des répertoires est significatif, ce qui permet de choisir par défaut une version donnée d'une commande. (Si plusieurs répertoires : le premier sera choisi)

PS1 : invite (*prompt*) de commande (par défaut \$)

PS2 : invite de continuation (>).

PS3: Invite de select

LOGNAME ou USER : nom de connexion

SHELL : nom du shell utilisé

PWD : répertoire courant

Protection

Protection d'une variable avec ***readonly***

- *toute modification interdite*
- *toute suppression (unset) impossible*

Syntaxe : `readonly var1`

Sans argument, `readonly` affiche la liste des variables protégées

Opérations sur les variables

Substitution

a=oui

b=\$a ▯ b vaut oui

Concaténation : juxtaposition simple s'il n'y a pas d'ambiguïté ou en entourant le nom de la variable par des accolades (`${var}`)

Exemples :

a=Bon b=jour ▯ affectation de a et de b

echo \$a\$b ▯ Bonjour

echo \$b1 ▯ rien: b1 non définie

echo \${b}1 ▯ jour1

Opérations arithmétiques

(use *let* on bash not zsh)

commande let:

let 5+9 (calculée mais on ne peut pas la récupérer car on n'a pas son adresse memoire)

```
let var=5+9
```

```
echo $var    --> 14
```

```
let var=5/9
```

```
echo $var    --> 0 (division en int)
```

```
v1=5
```

```
v2=6
```

```
let var=$v1*$v2 (sans espaces)
```

```
echo $var    --> 30
```

Opérations arithmétiques

En shell de Bourne, pour toute opération arithmétique ou logique sur une variable : commande `expr` d'UNIX

```
a=1
```

```
a='expr $a + 3' --> a=4
```

Korn shell et bash: nouvelle fonction `let` :
let var="val1 op val2"

vali : constante ou variable

op = opération arithmétique (+, -, *, /, %) ou logique (! (non)) ou comparaison (<, <=, >, >=, ==, !=)

- a=1
- let a="\$a+1" --> a=2
- let a="\$a >5" --> a=1 (faux)

Paramètres d'un script shell

Initialisés lors de l'exécution d'un shell-script

- \$# nombre d'arguments
- \$0 nom du shell-script
- \$i ième argument avec i = 1, 2 ... 9
- \$* ensemble des arguments
- \$\$ numéro (PID) du processus
- \$! numéro (PID) du dernier processus différé

- \$? code de retour de la dernière commande

Exemple : soit le fichier demo

```
echo "Nb Parametres = $#"  
echo "Parametre 1 = $1"  
echo "Liste = $*"
```

Exécution : demo un 2 trois

Nb Parametres = 3

Parametre 1 = un

Liste = un 2 trois

- Décalage ($\$2 \rightarrow \1 , $\$3 \rightarrow \2 , ...) : ***shift***
- Décalage de n ($\$n+1 \rightarrow \1 , ...) : ***shift n***
- Le paramètre \$? contient le code de retour de la dernière commande

Affectation de paramètres

Les paramètres \$1, ...\$# sont normalement affectés lors de l'appel d'un shell-script

Ils peuvent l'être aussi par la fonction **set : set arg1 [arg2 ...]**

Exemples :

- `set a b c` --> `$1=a, $2=b, $3=c, $# = 3, $* = a b c`

Quottes, apostrophe, guillemets et apostrophe inversée

Simple quote ou apostrophe

Les simples quotes délimitent une chaîne de caractères. Même si cette chaîne contient des commandes ou des variables Shell, celles-ci **ne seront pas interprétées**.

```
$variable=" ensa"
```

```
$ echo 'Mon mot de passe est $variable.'  
Mon mot de passe est $variable.
```

Doubles quotes ou guillemets

Les doubles quotes délimitent une chaîne de caractères, mais les noms de variable **sont interprétés par le shell**

```
$variable=" ensa"
```

```
$ echo 'Mon mot de passe est $variable.'  
Mon mot de passe est ensa.
```

Back-quote, apostrophe inversée

les Back-quotes délimitent une commande à exécuter. Les noms de variable et les commandes sont donc interprétés.

```
$echo `variable=« ensa`; echo "Mon mot de passe est $variable."`  
Mon mot de passe est ensa.
```

Autre utilisation

Mettre le résultat d'une commande dans une variable

```
$list=`ls`
```

```
echo $list -> même résultat de ls
```

Fonction test

Syntaxe : test expression ou [expression]

La fonction test évalue l'expression puis

- si elle est vraie, retourne VRAI (0)
- si elle est fausse, retourne FAUX (#0)

3 types de tests :

- attributs de fichiers
- variables
- entiers

Test des attributs de fichiers : **-type fichier**

- type = r, w, x
- ou type = f, d, c, b, p
- ou type = u(suid), g (sgid) k (sticky)
- ou type = s (taille > 0)

Exemple : test -x /ensa ou [-x /ensa]

Test des variables :

- -z var variable de longueur nulle
- -n var variable de longueur non nulle
- var1 = var2 pour var1 et var2 identiques
- var1 != var2 pour var1 et var2 différentes

Exemple : test \$rep = "q" ou [\$rep = "q"]

Test des entiers : **n1 cmp n2**

- `cmp = -eq (n1= n2),`
- `cmp = -ne (n1 # n2)`
- `cmp = -gt (n1 > n2)`
- `cmp = -ge (n1>= n2)`
- `cmp = -lt (n1 < n2)`
- `cmp = -le (n1<= n2)`

Exemple : **test \$# -gt 0 ou [\$# -gt 0]**

Possibilité :

- de grouper des expressions entre parenthèses
- de les combiner avec les opérateurs logiques : -o (OU), -a (ET), ! (NON)

Structures de contrôle

La commande if:

```
if commande  
then  
commandes  
fi
```

```
if commande  
then  
commandes 1  
else  
commandes 2  
fi
```

```
if commande1  
then commandes 1  
elif commande2  
then commandes 2  
else commandes 3  
fi
```

Affichage du type du fichier fic1

```
if [ -f fic1 ]  
    then  
        echo "fic1 : fichier"  
elif [ -d fic1 ]  
    then  
        echo "fic1 : répertoire"  
else  
        echo "fic1 : spécial"  
fi
```

La boucle for:

```
for var in listeDeValeur  
do  
Cmds  
Done
```

Copie d'un fichier fic1 sous les répertoires /users/rep1 à /users/rep4

```
for n in 1 2 3 4  
do  
cp fic1 /users/rep$n  
done
```

La boucle while:

```
while commande  
do  
    cmds  
Done
```

Affichage de tous les paramètres un à un

```
while [ $# -gt 0 ]  
do  
    echo "Paramètre = $1"  
    shift  
done
```

Structure case

```
case mot in  
    motif1) commandes 1;;  
    motif2) commandes 2;;  
    ...  
    motifn) commandes n;;  
esac
```

Exemple : récupération dans une variable dir du répertoire courant ou du premier paramètre

```
case $# in
  0) dir=`pwd`;
  1) dir=$1;;
  *) echo "Erreur de syntaxe"
     exit;;
esac
```

La boucle until: *Jusqu'à ce que*

```
until cmd  
do  
  commandes  
done
```

Exemple : Affichage des entiers de 1 à 9

```
i=1  
until [ "$i"=10 ]  
do  
  echo $i  
  let i=$i+1  
done
```

Alias

L'alias est le mécanisme qui permet de désigner une commande par un nouveau nom.

```
$ alias toto='ls -l'      # définition
```

```
$ toto                    # utilisation
```

```
total 4
```

```
-rw-r--r-- 1 ensa 42 Nov 10 09:44 liste1
```

```
-rw-r--r-- 1 ensa 23 Nov 10 09:43 file
```

```
-rw-r--r-- 1 ensa 29 Nov 10 09:40 ff
```

```
-rw-r--r-- 1 ensa 171 Nov 10 09:40 tata
```

```
$ toto >listefile
```

```
$ alias                # liste tous les alias  
toto='ls -l'
```

```
$ unalias toto          # suppression d'alias  
$ toto                  # commande inexistante  
toto: not found
```

Lecture d'une ligne sur stdin (clavier)

`read var1 var1 ... varn`

Séparateur : **IFS**

Exemple:

```
$ read a b c
```

```
$ Ensa est une ecole      # frappe clavier
```

```
Résultat: a=ensa b=est c=une ecole
```

Variante bash pour afficher une invite : **option -p**

```
Exemple: read -p "Quel temps? " a b c
```

```
Quel temps?      # lecture clavier
```

Fonction eval

Syntaxe : eval commande

eval permet une double évaluation

Elle permet d'accéder au contenu du contenu d'une variable

Exemple :

Fonctions

Les fonctions sont des bouts de scripts nommés, directement appelés par leur nom, pouvant accepter des paramètres.

Définition

```
nom()  
{  
  commandes  
}
```

Exécution : nom [par1 par2 ...]

Les paramètres sont utilisables comme pour un shell-script avec \$1 ... \$9, \$# et \$*

Fonction "point "

L'exécution d'un shell script s'effectue dans le cadre **d'un processus fils**

La fonction `.` (point espace) permet de l'exécuter dans le même processus , *même environnement (variables, répertoire courant...)*

Exemple: si un fichier fonctions contient une liste de déclarations de fonctions, on peut intégrer celles-ci dans le processus courant avec :

- `.` fonctions

Boucle while sur lecture fichier :

while read x	ou	cat fichier while read x
do	do	
commandes		commandes
done <fichier	done	